

First Week- Gen AI Assignment

1) Find 10 different model providers and their models released in last 12 months

Details to collect:

Model Provider | Model | Release Date | Cut off Date | Cost of tokens | Context Window | Type | When to use this model |

Here’s a structured list of 10 major AI models released in the last 12 months (July 2025 – July 2026), across different providers, with details on release date, cutoff, pricing, context window, type, and best use cases.

Comparison of 10 AI Models (2025–2026)

Provider	Model	Release Date	Cutoff Date	Cost (per 1M tokens)	Context Window	Type	Best Use Case
OpenAI	GPT-5.6 Sol	Jul 9, 2026	2026	\$5 in / \$30 out	1M	Frontier reasoning	High-stakes reasoning, enterprise AI
OpenAI	GPT-5.5 Pro	Apr 24, 2026	2026	\$30 in / \$180 out	1M	Frontier	Advanced research, coding agents
Anthropic	Claude Sonnet 5	Jul 1, 2026	2026	\$2 in / \$10 out	1M	General reasoning	Balanced cost/performance for enterprise
Anthropic	Claude Opus 4.8	May 28, 2026	2026	\$5 in / \$25 out	1M	Frontier	Complex reasoning, safety-critical tasks
Google DeepMind	Gemini 3.5 Flash	May 19, 2026	2026	\$1.50 in / \$9 out	1M	Frontier	Fast agentic reasoning, multimodal
Google	Gemma 4 (12B)	Jun 3, 2026	2026	Free (open weights)	131K	Open-source	Lightweight reasoning, research use
Meta	Llama 3.3 (70B)	Dec 2025	2025	Free (open weights)	128K	Open-source	Open research, tool use, coding
Mistral AI	Mistral Medium 3.5	Apr 29, 2026	2026	\$1.50 in / \$7.50 out	256K	Open frontier	Coding + agentic tasks, open deployment
xAI	Grok 4.3	Apr 2026	2026	\$1.25 in / \$2.50 out	1M	Frontier	Real-time data via X/Twitter
DeepSeek	DeepSeek V4-Pro	Apr 24, 2026	2026	\$0.435 in / \$0.87 out	1M	Open frontier	Cost-efficient reasoning, open deployment

Key Insights

- **Context windows have expanded dramatically:** Most frontier models now support **1M tokens**, enabling book-length context and complex workflows.
- **Pricing tiers vary widely:** OpenAI's GPT-5.5 Pro is the most expensive (\$180 per 1M output tokens), while DeepSeek V4-Pro is among the cheapest (<\$1 per 1M).
- **Open vs Closed:** Meta's Llama 3.3 and Google's Gemma 4 are **open-source**, ideal for research and customization. OpenAI and Anthropic models remain **API-only**.
- **Specialization:**
 - **Claude Sonnet 5** → balanced cost/performance for enterprise.
 - **Gemini 3.5 Flash** → speed + multimodality.
 - **DeepSeek V4-Pro** → budget-friendly open deployment.
 - **Grok 4.3** → real-time data integration.

Risks & Trade-offs

- **Cost vs capability:** Frontier closed models (GPT-5.5 Pro, Claude Opus 4.8) are powerful but expensive.
 - **Open models** (Llama, Gemma, DeepSeek) are cheaper and customizable but may lag in safety alignment.
 - **Cutoff dates:** Most models are trained up to **2026**, but real-time access is limited (only Grok integrates live data).
-

Recommendation:

- Use **Claude Sonnet 5** or **Gemini 3.5 Flash** for balanced enterprise tasks.
- Choose **DeepSeek V4-Pro** or **Llama 3.3** if you need **open-source, cost-efficient deployment**.
- Opt for **GPT-5.6 Sol** or **Claude Opus 4.8** for **mission-critical reasoning** where accuracy outweighs cost.

2) Read 2–3 articles about how transformer works?

Goal to understand:

(a) Self-attention mechanism

(b) Vector embedding

(c) Feed forward Neural Network

Transformers rely on three pillars—self-attention, embeddings, and feed-forward networks. Recent research shows that while attention is the headline innovation, embeddings and FFNs are equally critical for performance and interpretability.

Transformers rely on three pillars—self-attention, embeddings, and feed-forward networks. Recent research shows that while attention is the headline innovation, embeddings and FFNs are equally critical for performance and interpretability.

📄 Key Articles Compared

Here are two foundational and two recent perspectives:

Focus	Article	Main Contribution	Strengths	Limitations
Self-Attention	<i>Attention Is All You Need</i> (Vaswani et al., 2017, arXiv)	Introduced the Transformer, replacing recurrence with self-attention.	Clear mathematical formulation of Q/K/V, multi-head attention, parallelization benefits.	Early version, limited empirical scope (translation tasks).
Embeddings	<i>Understanding Transformers and Attention Mechanisms</i> (arXiv, 2024)	Explains tokenization → vectorization pipeline, embedding matrices, positional encodings.	Practical examples (e.g., Llama 3, Gemma vocab sizes), math clarity for applied users.	Focused on applied math, less on downstream performance.
Embedding Structure	<i>How to Dissect a Muppet: Transformer Embedding Spaces</i> (MIT Press, 2022)	Decomposes embeddings into contributions from attention, FFN, and biases.	Provides interpretability framework, shows FFN vs attention contributions differ by task.	More theoretical, less practitioner-friendly.
Feed-Forward Networks	<i>Attention Is Not All You Need: Importance of FFNs</i> (arXiv, 2025)	Shows FFNs carry more parameters than attention (8:3 ratio), experiments with deeper FFNs.	Empirical evidence that FFNs are vital, deeper FFNs improve efficiency.	Focused on decoder-only models, not encoders.

🔗 Self-Attention vs Embeddings vs FFN

- **Self-Attention:**
 - Captures long-range dependencies in constant steps.
 - Multi-head design allows different “views” of relationships.
 - Best for contextualizing words in flexible order.
 - **Vector Embeddings:**
 - Map discrete tokens into dense vectors.
 - Positional embeddings (APE, RoPE, ALiBi) add sequence order.
 - Embedding space structure influences generalization and fine-tuning.
 - **Feed-Forward Networks (FFN):**
 - Apply non-linear transformations after attention.
 - Act as “key-value memory,” storing task-specific patterns.
 - Often hold more parameters than attention, making them critical for scaling.
-

🔗 Comparative Insights

- **Attention dominates headlines**, but **FFNs carry more weight in parameters** and are essential for performance.
 - **Embeddings set the foundation**: without good token + positional embeddings, attention cannot contextualize properly.
 - **Interpretability studies** show embeddings are additive structures, with attention and FFN contributing differently depending on task.
 - **Recent experiments** suggest deeper FFNs (3-layer) outperform standard 2-layer ones, reducing training loss with fewer parameters.
-

Takeaway

- **Self-attention** = relational reasoning.
- **Embeddings** = semantic representation + order.
- **FFNs** = non-linear enrichment + memory.

Together, they form the “transformer block.” If you want to **optimize or interpret transformers**, don’t just focus on attention—embedding design and FFN depth are equally decisive.